

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

«Дополнительные потоки данных в файлах NTFS»

по дисциплине «Информационная безопасность»

Студент:

Анкудинов Кирилл Константинович

Кортыш Константин Олегович

Соколов Артём Олегович

Группа:

P3418

Преподаватель:

Оголюк Александр Александрович

Содержание

1. Ход работы	2
1.1. Создание дополнительного потока	2
1.2. Сравнение размера файла (п. 1.0)	2
1.3. Сравнение свободного места (п. 1.1)	2
1.4. Зависимость свободного места от объёма данных (п. 1.2)	3
1.5. Вывод потока в отдельный файл (п. 2.0)	4
1.6. Команды cmd.exe (п. 3.0)	5
1.7. Прикладные программы (п. 4.0)	5

1. Ход работы

Работа выполнена в Windows 10 версии 10.0.19045.6456, в cmd.exe. Файла command.com в системе нет. Несистемный том — D:, метка Kingston Flash, файловая система NTFS 3.1. Размер сектора 512 байт, размер кластера 4096 байт. Каталог экспериментов — D:\ads-lab. Свободное место здесь и далее — поле Total free bytes команды fsutil volume diskfree D:.

1.1. Создание дополнительного потока

Пустой файл создан командой `type nul > test.txt`. До записи в поток `dir` показывал размер 0 байт. Вывод `dir` перенаправлен в дополнительный поток, затем поток прочитан и открыт в Notepad:

```
dir > test.txt:aaa.txt
more < test.txt:aaa.txt
notepad test.txt:aaa.txt
```

`more` вывел текст листинга каталога D:\ads-lab на момент создания потока: файл `test.txt` размером 0 байт и 30 885 978 112 байт свободно. Notepad открыл именно этот поток, заголовок окна был `test.txt:aaa.txt - Notepad`. Основной поток при этом остался пустым: `type test.txt` ничего не печатает.

1.2. Сравнение размера файла (п. 1.0)

Состояние	Размер, байт	Как измерен
До создания потока	0	<code>dir</code>
После, основной поток	0	<code>dir</code>
После, поток <code>aaa.txt</code>	331	<code>dir /r</code>

Таблица 1. Размер файла до и после создания дополнительного потока

Обычный `dir` после создания потока по-прежнему показывает 0 байт: в эту цифру попадает только основной поток. `dir /r` отдельно показывает `test.txt:aaa.txt: $DATA` длиной 331 байт. Свободное место на томе при этом не изменилось и осталось 30 885 978 112 байт: поток такого размера ещё помещается в запись MFT и отдельный кластер не получает.

1.3. Сравнение свободного места (п. 1.1)

На `C:\Temp\payload.bin` записано 52 428 800 байт. Источник лежит на другом томе, поэтому расход места на D: относится только к потоку. Данные переданы перенаправлением, длина потока при этом сохранилась:

```
type C:\Temp\payload.bin > test.txt:big
```

Состояние	Свободно, байт
До записи в поток	30 885 978 112
После записи 52 428 800 байт	30 833 549 312
Разница	52 428 800

Таблица 2. Свободное место на D: до и после записи в поток

Разница совпала с длиной записанных данных. 52 428 800 делится на размер кластера без остатка (12 800 кластеров по 4096 байт), поэтому округления сверх этой длины не видно. `dir test.txt` по-прежнему показывает 0 байт, а `dir /r` — поток `big` длиной 52 428 800 байт рядом с потоком `aaa.txt` длиной 331 байт. Отображаемый размер файла и занятое место на томе расходятся: место уменьшается на длину потока, а размер в `dir` относится только к основному потоку.

1.4. Зависимость свободного места от объёма данных (п. 1.2)

На том же томе создан файл `measure.txt`. В его поток `data` по очереди записывались блоки известной длины. Каждый следующий блок заменял предыдущее содержимое потока:

```
type N.bin > measure.txt:data
```

Длина `N.bin` задавалась точно, без добавочного перевода строки. После каждой записи снимался остаток свободного места. Поток `test.txt:big` в это время не менялся, поэтому приращения в таблице относятся к `measure.txt:data`.

Записано, байт	Свободно, байт	Изменение, байт
0	30 833 549 312	0
100	30 833 549 312	0
500	30 833 549 312	0
900	30 833 545 216	4 096
2 000	30 833 545 216	0
4 095	30 833 545 216	0
4 096	30 833 545 216	0
4 097	30 833 541 120	4 096
8 192	30 833 541 120	0
12 288	30 833 537 024	4 096
65 536	30 833 483 776	53 248
1 048 576	30 832 500 736	983 040
4 194 304	30 829 355 008	3 145 728

Таблица 3. Свободное место на томе D: в зависимости от размера потока

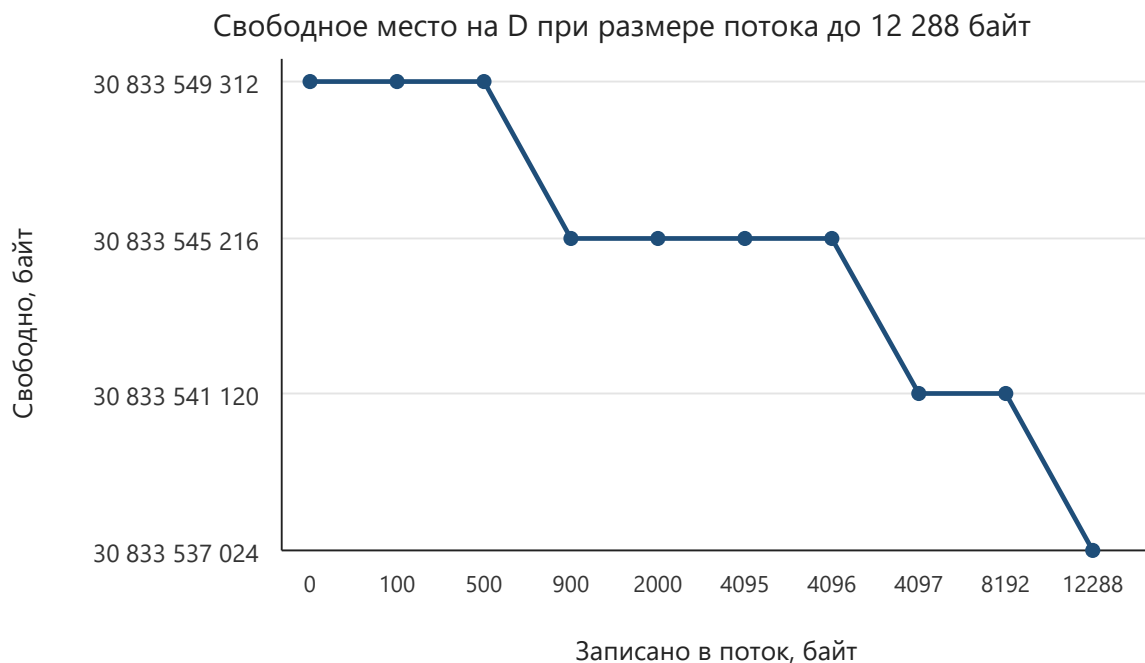


Рис. 1. Свободное место при размере потока до 12 288 байт. На полном диапазоне до 4 МиБ ступени в тысячи байт сливаются, они видны в таблице

Зависимость ступенчатая, шаг ступени равен кластеру, 4096 байт.

Пока поток короткий, место не меняется. При 100 и 500 байтах свободно те же 30 833 549 312 байт, что и при пустом потоке: данные лежат в записи MFT. На 900 байтах занят первый кластер, свободно становится на 4096 байт меньше. Дальше объём стоит на месте, пока хватает уже выделенных кластеров: 2000, 4095 и 4096 байт по-прежнему занимают один кластер. 4097-й байт занимает второй кластер, 12 288 байт — третий.

Крупные размеры укладываются в ту же схему. 65 536 байт — это 16 кластеров; от трёх кластеров на предыдущем шаге прибавилось $13 \times 4096 = 53\,248$ байт. 1 048 576 байт — 256 кластеров, прирост $240 \times 4096 = 983\,040$ байт. 4 194 304 байта — 1024 кластера, прирост $768 \times 4096 = 3\,145\,728$ байт. Итоговое уменьшение свободного места от пустого потока до 4 194 304 байт равно 4 194 304 байтам: длина кратна кластеру, лишнего кластера нет.

1.5. Вывод потока в отдельный файл (п. 2.0)

```
more < test.txt:aaa.txt > testdir.txt
```

Файл `testdir.txt` получил те же 331 байт, что и поток `aaa.txt`, и тот же листинг каталога. Повторная выгрузка потока во второй файл и сравнение `fc /b` дали `FC: no differences encountered`.

Напрямую сравнить поток и файл команда `fc` не смогла: `FC: cannot open TEST.TXT:AAA.TXT - No such file or folder`. Путь с именем потока она не открывает, поэтому сравнивались уже выгруженные файлы.

1.6. Команды `cmd.exe` (п. 3.0)

Команда	Результат
<code>more < file:stream</code>	Печатает содержимое потока
<code>type file > file:stream</code>	Записывает данные в поток, длина сохраняется
<code>type file:stream</code>	Ошибка синтаксиса, поток как аргумент не принимается
<code>echo text> file:stream</code>	Создаёт или перезаписывает поток
<code>echo text>> file:stream</code>	Дописывает в конец потока
<code>copy</code> в имя с двоеточием	Ошибка синтаксиса, 0 файлов. И как источник, и как назначение
<code>copy file dest</code>	Копирует файл вместе с потоками. Проверено на D: и при копировании на C:
<code>dir</code>	Показывает только размер основного потока
<code>dir /r</code>	Показывает каждый поток и его длину
<code>ren</code>	Переименование сохраняет потоки
<code>del file:stream</code>	Ошибка синтаксиса, поток остаётся
<code>del file</code>	Удаляет файл вместе со всеми потоками
<code>fc file:stream</code>	Не открывает путь с двоеточием

Таблица 4. Работа команд `cmd.exe` с дополнительными потоками

Запись и чтение потока в `cmd.exe` проходят через перенаправление: `>`, `>>` и `more <`. Команды, которым имя потока передано аргументом (`type`, `copy`, `del`, `fc`), отвергают двоеточие. Исключение — `dir /r`, который потоки показывает. Обычный `copy` файла на NTFS, без двоеточия в аргументе, потоки переносит: после `copy cmd-test.txt copy.txt` у копии был тот же поток `echo.txt`, в том числе у копии на диске C:.

1.7. Прикладные программы (п. 4.0)

Программы запускались с путём `D:\ads-lab\test.txt:aaa.txt`. Hiew в системе нет, вместо него проверены WordPad и Microsoft Word.

Программа	Результат
Notepad	Открыл поток. Заголовок окна: test.txt:aaa.txt - Notepad
WordPad	Открыл поток. Заголовок окна: test.txt:aaa.txt - WordPad
write.exe	Запускает WordPad, и уже WordPad открывает поток
Microsoft Word	Поток не открыл. Появился пустой документ «Документ1»

Таблица 5. Работа прикладных программ с дополнительным потоком

Notepad и WordPad понимают синтаксис файл:поток и читают указанный поток, а не основной. Word этот синтаксис игнорирует и открывает новый пустой документ.